

Vorlesung 2: Anforderungen und Prototypen

Prof. Dr. Falk Howar

Automated Quality Assurance
Lehrstuhl für Software Engineering
Fakultät für Informatik
TU Dortmund



Anforderungen.

- Herausforderungen und Methoden im Bereich der Erhebung, Dokumentation und Management von Anforderungen **aufzählen** und **erläutern**



- Detaillierungsgrade von Anforderungen **erläutern**, **auf Eignung prüfen** und bei der Anforderungsanalyse **nutzen**



- Qualitätskriterien auf Anforderungen **anwenden**, um diese zu **bewerten**



- Methoden zum Erheben und Validieren von Anforderungen **anwenden**



- Grundlegendes **Verständnis** des Design Thinking Vorgehens



- Die vorgestellten Methoden (User Stories, Personas, Experteninterviews) **abwägend einsetzen** können



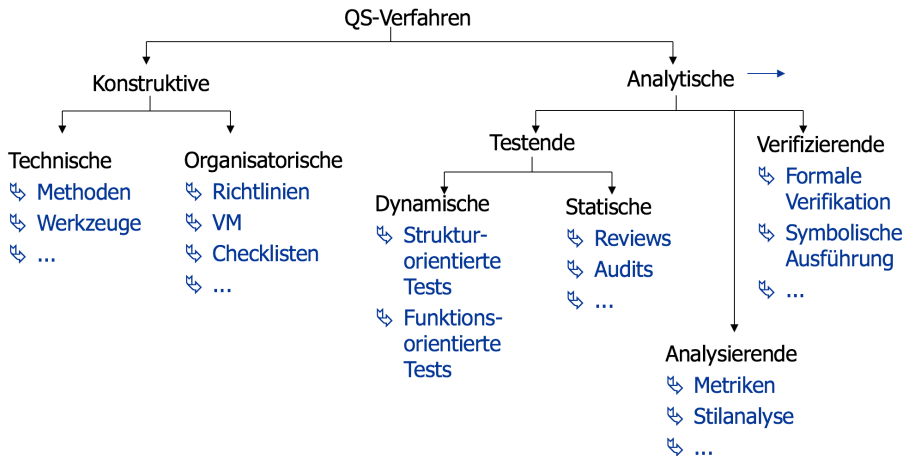
- Arten von Prototypen **benennen** und **abwägend einsetzen** können



Agenda

1. Motivation
2. Begriffe und Definitionen
3. Erheben von Anforderungen
4. Dokumentation von Anforderungen
5. Qualitätssicherung und Management
6. Management / Werkzeuge
7. Literatur und Referenzen

Klassifizierung von Ansätzen



Beispiel 1: Instant Messaging Start-Up IMVU

Initiale Annahmen:

- 1 Nutzer von Instant Messaging haben Interesse an 3D Chat
- 2 Nutzer haben starke Bindung an existierende Messaging Clients



Erste Version:

- IMVU baut Add-On für existierende Messaging Systeme
- Erste Version nach 25.000 Zeilen Code

Initialer Misserfolg:

- Add-On wird nicht genutzt
- Zweifel an Validität der Annahmen

Eric Ries, The Lean Start-Up

<https://www.slideshare.net/venturehacks/the-lean-startup-2/13-IMVU>

Beispiel 1: IMVU

Akzeptanztests:

- Tester haben Spaß beim Erstellen von Avataren
- Tester geben an, dass sie Produkt nicht an Freunde empfehlen würden:
"No way" - **unsicher ob es cool ist.**
- **Aber:** Tester wollen im 3D Chat neue Freunde zu finden ohne soziales Risiko durch eigenes Bewerben des Produkts



<https://www.mmogratis.com/imgs/IMVU-gs1.jpg>

Beispiel 1: IMVU

Markt Analyse zeigt:

- Interessierte Kunden nutzen schon bis zu 7(!) Messaging Apps

Revidierte Annahmen:

- ① Nutzer von Instant Messaging haben Interesse an 3D Chat
- ② Nutzer haben **keine** starke Bindung an existierende Messaging Clients

Neue Version:

- Eigenständiges 3D Messaging System und soziales Netzwerk
- Technisch kleine Änderung hat großen Einfluss auf Akzeptanz



An online community where members use 3D avatars to meet new people, chat, create and have fun with their friends



Beispiel 2: Überlingen (2002)

- Mid-Air Kollision von zwei Flugzeugen in der 71 Menschen sterben
- Beide Flugzeuge ausgestattet mit Collision Avoidance System (TCAS)
- TCAS: Onboard System, das bei Gefahr eines sog. Loss of Separation (LOS) Ausweichempfehlungen gibt
- **Ursache (u.A.):**
Fehlende Anforderungen an TCAS
- **Folgen:**
TCAS II hat sog. 'Sense Reversal' Ausweichempfehlungen



- Beide Flugzeuge auf Höhe FL360
- TCAS instruiert TU-154 zu steigen, Boeing 757 zu sinken
- Fluglotse instruiert TU-154 zu sinken und TU-154 folgt Anweisung des Fluglotsen
- Beide Flugzeuge sinken

<http://www.1001crash.com>

<https://www.youtube.com/watch?v=rycXgXPuDWU>

https://en.wikipedia.org/wiki/2002_%C3%9Cberlingen_mid-air_collision

Projekte scheitern wegen fehlender/schlechter Anforderungen

Project Impaired Factors	% of Responses
1. Incomplete Requirements	13.1%
2. Lack of User Involvement	12.4%
3. Lack of Resources	10.6%
4. Unrealistic Expectations	9.9%
5. Lack of Executive Support	9.3%
6. Changing Requirements & Specifications	8.7%
7. Lack of Planning	8.1%
8. Didn't Need It Any Longer	7.5%
9. Lack of IT Management	6.2%
10. Technology Illiteracy	4.3%
Other	9.9%

100%
> 40%

nach <https://www.projectsmart.co.uk/white-papers/chaos-report.pdf>

Erfolgsfaktoren in Projekten

Project Success Factors	% of Responses
1. User Involvement	15.9%
2. Executive Management Support	13.9%
3. Clear Statement of Requirements	13.0%
4. Proper Planning	9.6%
5. Realistic Expectations	8.2%
6. Smaller Project Milestones	7.7%
7. Competent Staff	7.2%
8. Ownership	5.3%
9. Clear Vision & Objectives	2.9%
10. Hard-Working, Focused Staff	2.4%
Other	13.9%

100%
> 35%

nach <https://www.projectsmart.co.uk/white-papers/chaos-report.pdf>

Motivation.

“Es ist wichtig, die Wünsche und Bedürfnisse der Stakeholder zu verstehen, zu dokumentieren sowie die Anforderungen zu spezifizieren und zu managen, um das Risiko zu minimieren, dass das System nicht den Wünschen und Bedürfnissen der Stakeholder entspricht.” [1]

Vorgehen / Methoden.

- Woher wissen wir, was die Software leisten soll?
- Wie beschreiben wir, was die Software leisten soll?

Qualität.

- Haben wir alle (wichtigen) Anforderungen erfasst?
- Haben wir die Anforderungen verständlich dokumentiert?

SWK Anwendungsbeispiel

Remote Home Control

- Geschäftsidee: Kunden können Geräte im Haus kontrollieren und steuern.
- Größeres Software Projekt: Sensoren, Aktuatoren, Server, App, ...

Von der Idee zum System

- [WARUM] Was ist der Nutzen?
- [WAS] Welche Anforderungen muss es erfüllen?



<https://flic.kr/p/oStisy> (CC BY 2.0)

Remote Home Control - Brainstorming

- Welches Problem kann so ein System für Euch lösen?
- Welche Features bräuchte es dafür?

Agenda

1. Motivation
2. Begriffe und Definitionen
3. Erheben von Anforderungen
4. Dokumentation von Anforderungen
5. Qualitätssicherung und Management
6. Management / Werkzeuge
7. Literatur und Referenzen

Zuerst klassisch, dann agil!

Anforderungen

Definition (Anforderung)

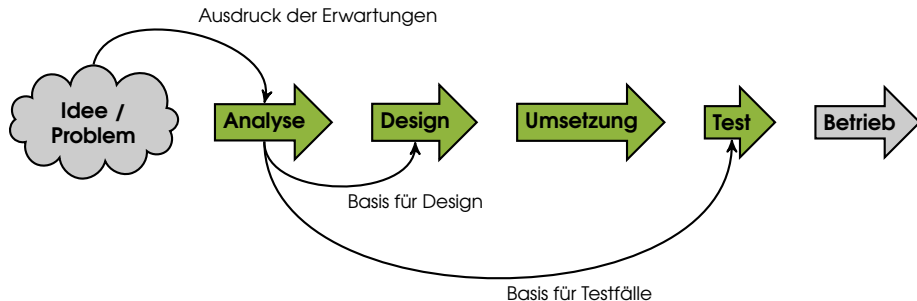
Eine Anforderung ist:

- ① Eine Bedingung oder Fähigkeit, die von einem Benutzer (Person oder System) zur Lösung eines Problems oder zur Erreichung eines Ziels benötigt wird.
- ② Eine Bedingung oder Fähigkeit, die ein System oder Teilsystem erfüllen oder besitzen muss, um einen Vertrag, eine Norm, eine Spezifikation oder andere, formell vorgegebene Dokumente zu erfüllen.
- ③ Eine dokumentierte Repräsentation einer Bedingung oder Eigenschaft gemäß (1) oder (2).

Beispiel:

“Der Benutzer kann das Licht in seinem Haus vom Mobiltelefon aus einschalten und ausschalten.”

Anforderungen im Entwicklungsprozess



Anforderungen als Basis von Verträgen



- **Lastenheft:**
Anforderungen des Auftraggebers
- **Pflichtenheft:**
Beschreibung des zu entwickelnden Systems durch potentiellen Auftragnehmer
 - Basis für Verträge über Software
 - Alle Anforderungen müssen überprüfbar sein
 - Ein objektiver Dritter muss entscheiden können
 - Notfalls Abnahme vor Gericht durchsetzen

Projektbetroffene (Stakeholder)

Definition (Stakeholder)

Projektbetroffene (Stakeholder) sind Einzelpersonen und Organisationen, die aktiv an einem Projekt beteiligt sind oder deren Interessen als Folge der Projektdurchführung oder des Projektabschlusses positiv oder negativ beeinflusst werden können.

Bei Softwareprojekten u. A.:

- die Entwickler
- der Kunde
- die Abteilung, die das Produkt später übernehmen und betreiben wird
- das Management

(vgl. PMBOK)

Systemgrenze und Kontext

Definition (Systemgrenze)

Die Systemgrenze trennt das geplante System von seiner Umgebung. Sie grenzt den im Rahmen des Entwicklungsprozesses gestaltbaren und veränderbaren Teil der Realität von Aspekten in der Umgebung ab, die durch den Entwicklungsprozess nicht verändert werden können.

Definition (Systemkontext)

Der Systemkontext ist der Teil der Umgebung eines Systems, der für die Definition und das Verständnis der Anforderungen des betrachteten Systems relevant ist.

Beispiele für Systemkontext:

- Personen (Stakeholder oder Stakeholdergruppen)
- Systeme im Betrieb (andere technische Systeme oder Hardware)
- Prozesse (z.B. Geschäftsprozesse)
- Dokumente (z.B. Gesetze, Standards)

Requirements Engineering

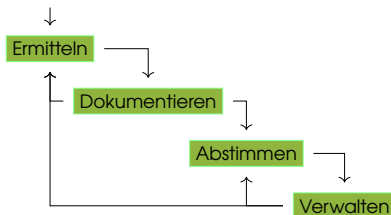
Definition (Requirements Engineering)

Requirements Engineering ist ein systematischer Ansatz zur Spezifikation und zum Management von Anforderungen mit den folgenden Zielen:

- ① Die relevanten Anforderungen zu kennen,
- ② Konsens unter den Stakeholdern über die Anforderungen herzustellen,
- ③ die Anforderungen konform zu vorgegebenen Standards zu dokumentieren und
- ④ die Anforderungen systematisch zu managen.

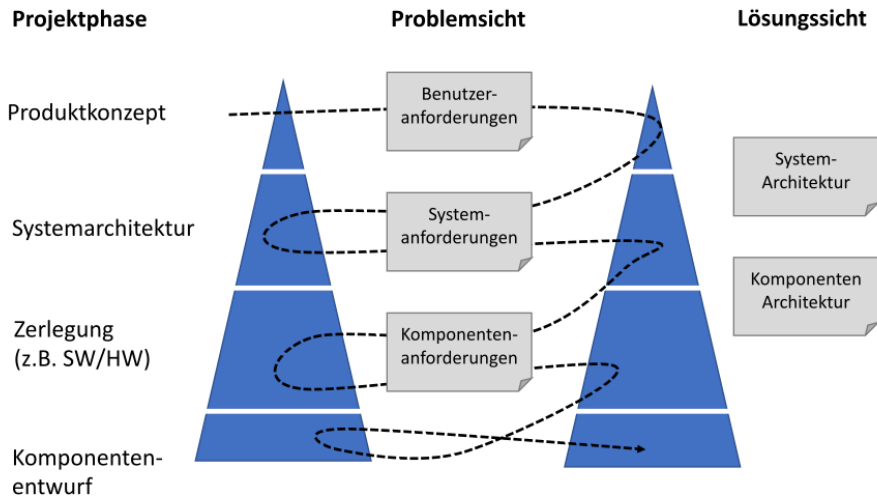
Tätigkeiten:

- Stakeholder und Anforderungen ermitteln
- Anforderungen dokumentieren
- Anforderungen prüfen und abstimmen
- Anforderungen verwalten



Def. nach Pohl, Rupp 2015: Basiswissen Requirements Engineering

Ko-Entwicklung von Architektur und Anforderungen



nach Ebert, 2014: Systematisches Requirements Engineering, 5. Auflage

Arten von Anforderungen

- Es gibt verschiedene Aspekte auf die Anforderungen sich beziehen können bzw. verschiedene Quellen für Anforderungen
- Quellen: Gesetze, Standards, Regeln in der Firma, Stakeholder, etc.
- Aspekte: Funktion, Performanz, Entwicklungsprozess, Werkzeuge, etc.
- Verschiedene Klassifikationsmodelle in der Literatur
 - SWK verwendet Schema nach Pohl, Rupp 2015: Basiswissen Requirements Engineering
 - Sommerville, 2012: Software Engineering benutzt feinere Kategorisierung

Funktionale Anforderungen

Definition (Funktionale Anforderung)

Eine funktionale Anforderung ist eine Anforderung an das Verhalten, das von einer Funktion des Systems bereitgestellt werden soll.

Beispiele:

- *"Der Benutzer kann sich mit seinem Nutzernamen oder seiner E-Mail Adresse anmelden."*
- *"Das System zeigt in der Übersicht zu jedem Gerät im Haus den Status und den Zeitstempel der letzten Interaktion an."*

Qualitätsanforderungen

Definition (Qualitätsanforderung)

Eine Qualitätsanforderung ist eine Anforderung, die sich auf ein Qualitätsmerkmal bezieht, das nicht durch funktionale Anforderungen abgedeckt wird.

Beispiele:

■ Performance:

"Das System muss jede Anfrage des Benutzers innerhalb von 30 Sekunden ausführen (auf System XY)."

"Der Speicherbedarf darf 512MB nicht übersteigen."

■ Zuverlässigkeit:

"Die Verfügbarkeit des Systems muss bei 99.999% liegen."

■ Benutzbarkeit:

„Das System muss die gespeicherten Objekte formatiert ausgeben können nach Schema XXXX.“

Randbedingungen

Definition (Randbedingungen)

Eine Randbedingung ist eine Anforderung, die den Lösungsraum jenseits dessen einschränkt, was notwendig ist, um die funktionalen Anforderungen und die Qualitätsanforderungen zu erfüllen.

Beispiele:

- **Anforderungen an den Entwicklungsprozess:** *„Der Entwickler muss mit dem Kunden monatliche Reviews der zu erstellenden Dokumente durchführen.“*
- **Umgebungsanforderungen:**
„Das System muss mit Java entwickelt werden und muss in der Sun Java VM Version XY laufen.“

- Bis hierhin: die "klassische" Sicht
- Fokus stark auf WAS und WIE (Systemsicht)!
- Erkenntnis aus agilem Vorgehen: es ist wichtig, das WARUM zu verstehen und zu adressieren (Nutzersicht)
- Das WAS und das WIE können sich dann ändern

Fokus auf Nutzer: Design

Design: **formgerechte** und **funktionale** Gestaltgebung und daraus sich ergebende Form eines Gebrauchsgegenstandes o. Ä.; Entwurf[szeichnung]

- Form eines Gebrauchsgegenstandes
- Funktional
- Entwurf

Designer*in: Person, die das Design von Gebrauchs- und Verbrauchsgütern entwirft (Berufsbezeichnung)

Agiles Vorgehen: Design Thinking

“Design thinking is a methodology for creative problem solving.”

Iteratives / nicht-lineares Vorgehen in fünf Phasen:

- ① Empathise: Einfühlen in Nutzer
- ② Define: Problem formulieren (z.B. **Persona** oder **User Story**)
- ③ Ideate: Radikale Ideen für Lösungen (z.B. **Brainstorming**)
- ④ Prototype: einfache **Prototypen** erstellen
- ⑤ Test: Mit Prototypen ausprobieren / **analysieren**

Agenda

1. Motivation
2. Begriffe und Definitionen
3. Erheben von Anforderungen
4. Dokumentation von Anforderungen
5. Qualitätssicherung und Management
6. Management / Werkzeuge
7. Literatur und Referenzen



How the customer explained it



How the project leader understood it



How the analyst designed it



How the programmer wrote it



What the beta testers received



How the business consultant described it



How the project was documented



What operations installed



How the customer was billed



How it was supported



What marketing advertised
iSwing



What the customer really needed

Ziel und Herausforderung

Ziel:

- Die relevanten Anforderungen kennen

Herausforderungen:

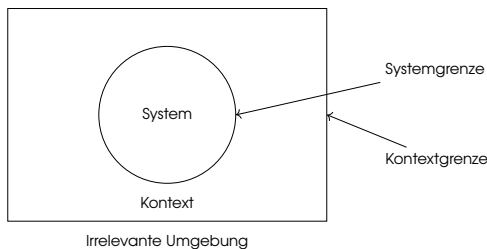
- Stakeholder haben oft implizites Wissen und unbewusste Erwartungen / Vorstellungen
- Kommunikation ist oft unvollständig und ungenau
- Gemeinsames Verständnis muss erst hergestellt werden
- Zugriff auf Stakeholder nicht immer einfach

Vorgehen

- ① Systemgrenze und Kontext ermitteln
- ② Relevante Datenquellen finden (bestehende Software, Gesetze, Normen)
- ③ Angemessene Methoden zum Erheben von Anforderungen auswählen
- ④ Anforderungen von/mit Stakeholdern erheben

Systemgrenze und Kontext identifizieren

Grenzen initial unscharf. RE schärft und verschiebt Grenzen.



- Wo kommen später Daten her? Wo fließen Daten hin?
- Wo werden Schnittstellen benötigt?
- Welche Systeme / Personen interagieren mit dem System?
- Was kann im Entwicklungsprozess verändert werden, was nicht?

Methoden und Hilfsmittel (teilw. in SWT)

Methoden:

- Brainstorming
- Beobachtung
- Recherche
 - Marktanalyse
 - Dokumentenanalyse: Normen / Gesetze
 - Systemanalyse
- Befragung
 - Fragebögen
 - Interviews

Hilfsmittel:

- Modelle
- Mind-Maps
- Value Proposition Canvas
- SWOT Analyse

Weitere Methoden und detaillierte Darstellung in der Literatur

Agiles Vorgehen: Design Thinking

“Design thinking is a methodology for creative problem solving.”

Iteratives / nicht-lineares Vorgehen in fünf Phasen:

- ① **Empathise:** Einfühlen in Nutzer
- ② Define: Problem formulieren (z.B. **Persona** oder **User Story**)
- ③ **Ideate:** Radikale Ideen für Lösungen (z.B. **Brainstorming**)
- ④ **Prototype:** einfache **Prototypen** erstellen
- ⑤ Test: Mit Prototypen ausprobieren / **analysieren**

Beispiel: Leitfadeninterview / Experteninterview



- Geführtes Interviews mit vorbereitetem Gesprächsleitfaden
- Experten = Träger relevanten Wissens
- Z.B. Sachbearbeiter beim Kunden, fachliche Experten, Power-Anwender, ...

vgl. Kaiser, 2014

Bild: <https://www.flickr.com/photos/alper/9516543726/>

Lizenz: <https://creativecommons.org/licenses/by/2.0/>

Struktur / Vorgehen

Struktur:

- Forschungsfragen → Fragenkomplexe → Interviewfragen
- Möglichst offene Fragen

Vorgehen:

- ① Entwurf Leitfaden aus Forschungsfragen
- ② Pre-Test (Verständnis, Interesse, Ablauf, Struktur, Dauer, Technik)
- ③ Auswahl Experten: Welcher Experte verfügt über die relevanten Informationen?
Welcher dieser Experten ist am ehesten in der Lage, präzise Informationen zu geben?
Welcher dieser Experten ist am ehesten bereit und verfügbar, um diese Informationen zu geben?
- ④ Kontakt / Durchführung / Auszeichnung / Transskription Interviews
- ⑤ Verschriftlichung

Fragetypen

- Einführungsfragen (Raum geben): Wir untersuchen heute ... Wie sieht das in Ihrem Umfeld aus.
- Strukturierende Fragen (leiten Bereiche ein): Ihren letzten Punkt würde ich gerne unter einem weiteren Aspekt beleuchten: Welche verschiedenen Interessen gibt es hier aus Ihrer Sicht
- Direkte Fragen (Wissen abfragen): Können Sie einschätzen, wie ...?
- Indirekte Fragen (Bewertung anderer Positionen): Welche Erwartungen haben aus ihrer Sicht andere Stakeholder ...?
- Spezifizierende Fragen (Nachfragen bei neuem Wissen): Sie haben XYZ benannt. Können Sie das genauer beschreiben?
- Interpretierende Fragen (Denkmuster des Befragten): Gehen Sie davon aus, dass die neue Anwendung ...?

Auswertung: qualitative Inhaltsanalyse

- ① Festlegung des Umfangs des zu analysierenden Materials, Analyseeinheiten, Kategorien
- ② Kodieren: Verdichtung mittels Überschriften (Kategorien)
- ③ Kernaussagen identifizieren und Interviews zusammenführen
- ④ Speziell im Umfeld Anforderungen: Ableitung von Personas, User Stories, etc.

Beispiel: Fokusgruppe



- Workshop mit Stakeholdern
- Experten = Träger relevanten Wissens
- Z.B. Sachbearbeiter beim Kunden, fachliche Experten, Power-Anwender, ...

vgl. Jacobsen und Meyer, 2019

Bild: <https://www.flickr.com/photos/umag/33866328461/>

Lizenz: <https://creativecommons.org/publicdomain/mark/1.0/>

Fokusgruppe: Ablauf

- Eröffnung: Vorstellung, Vertrauen schaffen
 - Persönliches Vorstellen
 - In Zweier Gruppen Vorstellen
- Einleitung: Thema und Zielsetzung kommunizieren
 - Impulse setzen (Z.B. durch Demo oder Entwurf)
- Überleitung: Austausch verschiedene Perspektiven Teilnehmer
 - Dokumentation von Themen (Z.B Karten)
 - Priorisierung von Themen und Fragestellungen
- Schlüsselphase: Hauptteil zur Bearbeitung der Fragen
 - Kleingruppen
 - Moderiertes Plenum
- Zusammenfassung: Zentrale Erkenntnisse zusammen bündeln, persönliche Fazits

Prototyp-basierte Entwicklung

Definition (Software Prototyp)

Ein Prototyp ist ein vorläufiges Versuchsmodell für eine Anwendung. Prototypen sind:

- **unvollständig:** es werden nur bestimmte Aspekte umgesetzt,
 - **vereinfachend:** umgesetzte Aspekte haben nicht die volle Funktionalität,
 - **Modelle:** z.B. oft nicht implementiert (kein MVP!),
 - **Versuche:** sollen Fragen im Designprozess beantworten.
-
- **Low Fidelity:** niedriger Detailgrad, keine Funktionalität, wesentliche Aspekte des Designs, Papier / Folien / Bilder
 - **High Fidelity:** hoher Detailgrad, erlebbare Funktionalität, detailliertes Design, technische Umsetzung

Design = Aussehen + Funktion!

Prototypen: Ziele und Fragen

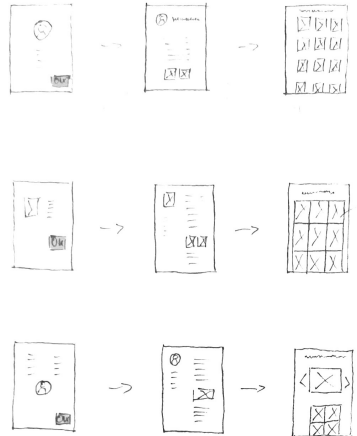
Testziele: Machbarkeit, Benutzbarkeit, UX, Wert/Preis, Ökonomischer Sinn

Arten:

- **Prototyp für Benutzer:** Simulation von Eigenschaften des Produkts für potentielle Nutzer.
- **Prototyp für technische Machbarkeit:** Implementierte Eigenschaften des Produkts.
- **“Life-Data” Prototype:** Prototyp der auf Produktivdaten arbeitet.

Scribbles (= Gekritzelt)

- Ansatz: Low-Fidelity Skizzen für Nutzer-Schnittstelle, Nutzerführung, Workflows, Seiten-Design, ...
- Ziel: Viele Varianten günstig, schnell und ohne Details erstellen
- Format: Papier, Tablet, Zeichenpapier



Wireframes

- Ansatz: "Strichzeichnungen" des Designs in Simulator
- Ziel: Simulation von Interaktion mit Produkt
- Format: Papier, interaktiv werkzeuggestützt am PC



Bildquelle: <https://www.sitepoint.com/how-writing-our-book-taught-us-about-killer-prototyping/>

Weitere Prototypen

Mock-Up:

- Statisches Modell des Produktes, technisch umgesetzt

Interaktiver Prototyp:

- Interaktives Modell des Produktes, technisch umgesetzt

Technische Prototypen

Technische Prototypen beantworten Fragen bezüglich der technischen Umsetzung einer Anwendung, z.B.,

- kann Bibliothek X genutzt werden, um die API zu generieren,
- welchen Durchsatz können wir erwarten, wenn wir MQTT / RPC / ... zur Kommunikation nutzen,
- ist das zentrale Element im Modell das Konto oder die Transaktion,
- wie können wir Software automatisch ausliefern wenn wir Gradle als Buildsystem nutzen?
- Github, Gitlab, Atlassian, oder ...?

Vorgehen: Aspekt recherchieren und technisch umsetzen mit klarem Evaluierungsziel, Konsequenzen dokumentieren, Entscheidung treffen.

“Life-Data” Prototypen

Implementierung eines wesentlichen Aspekts ohne die Implementierung/Qualität der Aspekte, die zu einem vollständigen Produkt oder Feature gehören.

- kann rein zur Datengewinnung /Entscheidungsfindung durch Entwickler ausgeführt werden, oder
- eingeschränkt für Nutzer verfügbar gemacht werden (Alpha, Beta, ...)

Beispiel:

Wir wollen wissen, ob es für BoardGame Inc. möglich ist, den Spielern sinnvolle und interessante Statistiken über ihre Spiele anzuzeigen.

- ① Algorithmen auf Daten aus echten Spielen anwenden
- ② Test-Nutzern Statistik anbieten

Agenda

1. Motivation
2. Begriffe und Definitionen
3. Erheben von Anforderungen
4. Dokumentation von Anforderungen
5. Qualitätssicherung und Management
6. Management / Werkzeuge
7. Literatur und Referenzen

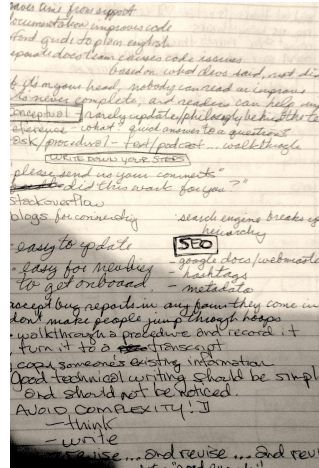
Ziel und Herausforderung

Ziel:

- Die relevanten Anforderungen verständlich und einheitlich dokumentieren

Ansätze:

- Sprache
 - Kann von allen Stakeholdern verstanden und genutzt werden
 - Oft unpräzise
- Modelle
 - Formale Syntax (und ggf. eindeutige Semantik)
 - Modelle abstrahieren



<https://flic.kr/p/dZ37gj>

Agiles Vorgehen: Design Thinking

“Design thinking is a methodology for creative problem solving.”

Iteratives / nicht-lineares Vorgehen in fünf Phasen:

- ① Empathise: Einfühlen in Nutzer
- ② **Define**: Problem formulieren (z.B. **Persona** oder **User Story**)
- ③ Ideate: Radikale Ideen für Lösungen (z.B. **Brainstorming**)
- ④ Prototype: einfache **Prototypen** erstellen
- ⑤ Test: Mit Prototypen ausprobieren / **analysieren**

Dokumentationsformen

Notation	Eigenschaften
Sätze in natürlicher Sprache	Nummerierte Sätze. Jeder Satz sollte eine Anforderung ausdrücken
Strukturierte natürliche Sprache (z.B. Standardformular oder Satz Schablonen)	Anforderungen sind in natürlicher Sprache verfasst. Sätze werden nach Template gebildet. Jeder Satzteil beschreibt einen Aspekt der Anforderung.
Formale Sprache	Es wird eine formale Sprache verwendet (ähnlich einer Programmiersprache). Beschrieben wird erwartetes Verhalten. (Wird heute selten benutzt)
Grafische Notation	Grafische Modelle, ergänzt um Text Annotationen, beschreiben die funktionalen Anforderungen an das System (z.B. UML Anforderungs- und Aktivitätsdiagramme)
Mathematische Spezifikation	Spezifikation basiert auf mathematischem Formalismus (Automaten oder Mengen). Eindeutig aber oft nicht nutzbar, da Kunden diese nicht verstehen.

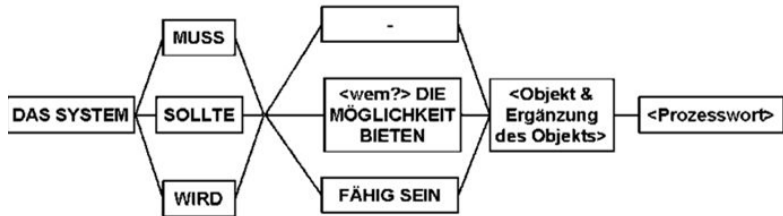
Detailliertere Darstellung: Sommerville, 2012

Natürliche Sprache

„Um Abhängigkeiten zwischen verschiedenen Requests des Fachmoduls zu vermeiden, darf das Fachmodul in der Kommunikation zum Intermediär kein Pipelining verwenden. Parallelität von Requests vom Fachmodul an den Intermediär VSDM kann ohne Pipelining über mehrere HTTP-connections erreicht werden.“ [VSDM-A_3066]

Strukturierte natürliche Sprache

Verbesserte Eindeutigkeit und Präzision durch (u.A. grammatikalische) Einschränkungen



Beispielsystem: Sophisten Satzschablonen

Quelle: Basiswissen Requirements Engineering (Pohl, Rupp)

Beispiel:

VSDM-A_3066: "(Das Fachmodul VSDM) DARF NICHT (Pipelining [...]) verwenden."

Formale Sprache

```
PROCESS Ground_level_estimation;  
IF (O.OOGROUND EQ $TRUE) THEN  
  G.ZGROUND = G.ZOWN;  
ELSEIF (G.RADAROUT GT P.RADARLOST) THEN  
  G.ZGROUND = -P.ZLARGE;  
ELSEIF (G.RADAROUT GT 0) THEN  
  ...  
END Ground_level_estimation;
```

"TCAS II requirements were defined by the Minimal Operational Performance Standard (MOPS) document. The MOPS was expressed using a combination of English and approximately 7,000 lines of lowlevel pseudocode."

Mats P.E.Heimdahl and Nancy G.Leveson and Jon D. Reese. **Experiences From Specifying the TCAS II Requirements Using RSML.** 17th Digital Avionics Systems Conference, 1998.

Modellierung

Unified Modeling Language (UML)

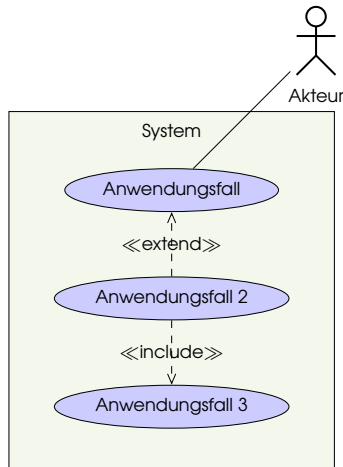
- Modellierung, Dokumentation, Spezifizierung und Visualisierung von komplexen Softwaresystemen.
- Standard der Object Management Group (OMG).
- Unterschiedliche Modellierungskonzepte auf einheitlicher Basis: 14 Diagrammtypen in Version 2.4.1
- Modell vs. Diagramm: UML-Modell besteht aus einem oder mehreren Diagrammen. Ein Diagramm entspricht einer bestimmten Sicht ("View") auf Modell (vgl. IEEE 1471).

Relevante Diagramme

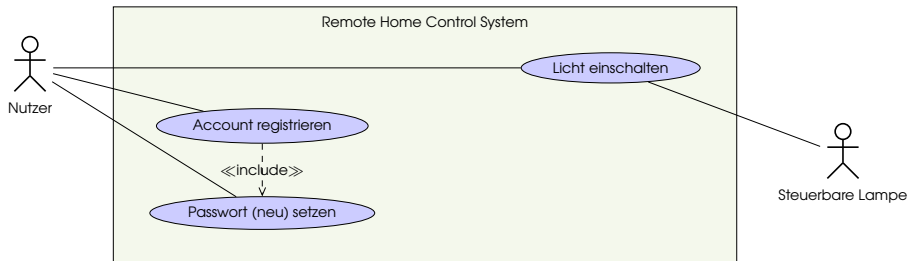
- **Anwendungsfalldiagramm**
High-Level Beschreibung der verschiedenen Anwendungsfälle und deren Beziehungen untereinander sowie zu Nutzen und anderen Systemen.
- **Klassendiagramm**
Struktur des Problembereichs (SWT / SoPra)
- **Aktivitätsdiagramm**
Darstellung von Abläufen von Aktivitäten, z.B. detaillierte Darstellung eines Anwendungsfalls
- **Zustandsdiagramm**
Ereignis-basierte Spezifikation von Reaktion des Systems auf Umgebung
(→ später in der Vorlesung)

UML Anwendungsfalldiagramme: Syntax

- Akteure: (System kann auch Akteur sein)
- Anwendungsfälle: Ellipsen mit Titel.
- Assoziationen: Linien zwischen Akteuren und Anwendungsfällen.
- Systemgrenzen: Rechtecke.
- include-Beziehungen: mit «include» gekennzeichnete gestrichelte Linie. Pfeil zum inkludierten Anwendungsfall
Inkludierter für den aufrufenden Anwendungsfall notwendig.
- extend-Beziehungen mit «extend» gekennzeichnete gestrichelter Linie. Pfeil vom erweiternden Anwendungsfall. Erweiternder kann vom aufrufenden Anwendungsfall aktiviert werden.



Beispiel: Anwendungsfalldiagramm



Mischformen: Schablone für Anwendungsfälle

In der Praxis findet man oft einen Mix verschiedener Dokumentationsmethoden

Beispiel:

- Top-Level: Anwendungsfall
- Zu jedem Anwendungsfall eine ausgefüllte Schablone mit Beschreibung des Anwendungsfalls und Varianten
- Unterstützung durch weitere UML Diagramme für mögliche Abläufe
- Referenz von relevanten Detail-Anforderungen, Qualitätsanforderungen oder UI-Anforderungen

Abschnitt	Inhalt
Bezeichner	UC-1
Name	Gerät schalten
Autoren	F. Howar
Priorität	...
Quelle	Nutzerbefragung
Verantwortlich	...
Kurz-Beschreibung	Der Nutzer schaltet ein Gerät im Haus von unterwegs ein.
Auslöser	Der Wunsch, ein Gerät zu schalten.
Vorbedingung	App installiert und Gerät registriert.
Nachbedingung	
Ergebnis	
Hauptszenario	App öffnen, Gerät aus Übersicht wählen, Gerät schalten
Alternativ-Szenarien	...
Ausnahme-Szenarien	...
Einzelne Anforderungen	REQ-F-10, ...
Qualitäten	REQ-Q-1, REQ-Q-2, ...

Fokus auf Stakeholder: Personas

Name Engelle Beruf Werkstatteinleiterin Alter 46

persönliche Bedürfnisse

- ungenutzte Angebote
- politischer Beistand
- wird viel und befindet
sich in der Leitung

Vorstellungen

- gleich gut mit 18
Jahren
- aktuelle Erfahrung
mit Online-Marketing
von Handwerksbetrieben

Pain Points

- Handwerksbetriebe-Flächen
sind immer in kleinen
Momenten am
abgesprochenen
Angebot (+3 bis 2)

Ziele der Person

Häufige Reden und
ein hoher Qualitäts-
anspruch gegen neue
Werkzeuge und
Kommunikation

Mediennutzung

- 60% mobil 40% stationär / Notebook
- druckt Artikel aus
- liest häufig online



Was sind Personas

- Fiktive Nutzer / Stakeholder. konkret aber Archetyp
- Ziele / Motivation
- "Pain Points", "Jobs to be Done"
- Szenarios (→ User Stories)
- Konsolidiert User Research Daten

Beispiel Strukturierte Sprache: User Stories

Als Coach will ich mein Profil einstellen.

- ☐ **Stundensatz, Verfügbarkeit und Standort sind Pflichtfelder**
- ☐ **Ein Coach kann seinen Lebenslauf als PDF hochladen**
- ☐ **Rolle ist Pflichtfeld: ScrumMaster, Product Owner, Coach**

- Dokumentationsform für Anforderungen (Cohn, 2004)
- Ursprünglich aus dem Bereich der agilen Methoden

Beispiel: User Stories

Feste Form: Als [Rolle] will ich [Ziel], damit [Grund für Ziel]

- Keine Systemdetails (oft noch unbekannt, bzw. verhandelbar)
- In der Sprache des Kunden bzw. der Domäne geschrieben
- Story Cards:
 - Vorderseite: User Story
 - Rückseite: Akzeptanzkriterien (nach Diskussion)
- Epics gruppieren einzelne Stories in Kontexten
- Können später durch Feature realisiert werden (Use Cases)

Dokumentation: User Stories (Beispiele)

- Als Scrum-Coach will ich mich anmelden, so dass ich mein Profil einstellen kann.
- Als Anbieter will ich nach Coaches suchen, so dass ich Kontakt aufnehmen kann.
- Als ehemaliger Kunde will ich einen Coach bewerten, so dass sich seine gute Arbeit herumspricht.

Qualität von User Stories

INVEST:

- **I für Independent:** Gegenseitig unabhängig voneinander
- **N für Negotiable:** Verhandelbar, also gestaltbar
- **V für Valuable:** Wert für den Kunden haben ($\rightarrow$ *Jobtobedone*, *BM*)
- **E für Estimable:** Schätzbar (Kosten für Umsetzung)
- **S für Small:** Kleiner Umfang
- **T für Testable:** Testbar um erfolgreiche Umsetzung zu kontrollieren

Agenda

1. Motivation
2. Begriffe und Definitionen
3. Erheben von Anforderungen
4. Dokumentation von Anforderungen
5. Qualitätssicherung und Management
6. Management / Werkzeuge
7. Literatur und Referenzen

Herausforderungen

■ Genauigkeit / Eindeutigkeit

- Kunden wissen nicht was sie wirklich wollen
- Kunden benutzen ihre eigene Fachsprache

■ Gemeinsame Erwartungen

- Verschiedene Beteiligte können widersprüchliche Anforderungen haben

■ Veränderung

- Anforderungen ändern sich während der Analyse und Entwicklung
- Neue Beteiligte mischen sich ein

Agiles Vorgehen: Design Thinking

“Design thinking is a methodology for creative problem solving.”

Iteratives / nicht-lineares Vorgehen in fünf Phasen:

- ① Empathise: Einfühlen in Nutzer
- ② Define: Problem formulieren (z.B. **Persona** oder **User Story**)
- ③ Ideate: Radikale Ideen für Lösungen (z.B. **Brainstorming**)
- ④ Prototype: einfache **Prototypen** erstellen
- ⑤ **Test**: Mit Prototypen ausprobieren / **analysieren**

Qualität von Anforderungen

Individuelle Anforderungen:

- Korrektheit:
Die Anforderung drückt die Erwartung der Betroffenen aus.
- Umsetzbarkeit:
Anforderung ist erfüllbar.
- Nachverfolgbarkeit:
Der Ursprung der Anforderung und ihre Entwicklung lassen sich feststellen.
- Eindeutigkeit:
Die Formulierung der Anforderung erlaubt nur eine gültige Interpretation.
- Prüfbarkeit:
Es lassen sich Kriterien oder Tests angeben, um nachzuweisen, ob die Anforderung erfüllt ist oder nicht.

Qualität von Anforderungen

Mengen von Anforderungen:

- Minimalität:
Es gibt keine doppelten oder irrelevanten Anforderungen.
- Konsistenz:
Die einzelnen Anforderungen widersprechen sich nicht.
- Vollständigkeit:
Die Anforderungen beschreiben das gewünschte System in allen relevanten Aspekten.
- Erweiterbarkeit / Anpassbarkeit:
Einzelne Anforderungen lassen sich unabhängig ändern.

Anforderungen validieren

Fragen

- Wird der Bedarf des Kunden vollständig abgedeckt?
- Verständlich formuliert?
- Konsistent mit anderen Anforderungen?
- Anforderung prüfbar?
- Änderbar ohne Einfluss auf andere Anforderungen?
- Realistisch mit Budget und Technologie?

Methoden

- Reviews
- Prototypen
- Erzeugung von Testfällen

Überprüfbarkeit von Qualitätsanforderungen

- Beschreibung von Qualitätsanforderungen oft unsystematisch und ungenau.
- Darauf achten, messbare Kriterien zu formulieren!

Eigenschaft	Messbare Formulierung
Geschwindigkeit	Ausgeführte Transaktionen / Sekunde Antwortzeit
Speicher	Mbytes Anzahl ROM
Benutzbarkeit	Einarbeitungszeit Umfang der Hilfe
Zuverlässigkeit	Durchschnittliche Zeit bis zu einem Absturz Wahrscheinlichkeit eines Ausfalls Fehlerhäufigkeit Verfügbarkeit
Robustheit	Zeit bis Wiederherstellung nach Ausfall Anteil fehlerhafter Vorgänge

Gruppierung als Basis der Qualitätssicherung

Bei mehreren tausend Anforderungen wird Struktur benötigt.

Sichten:

- Ausschnitte aus dem Anforderungsdokument für bestimmte Rollen (Entwickler, Kunde, etc.) erzeugen ("Sichten")

Kategorisieren:

- Art der Anforderung: Systemfunktionalität, Benutzerschnittstelle, Datenbank, Kommunikation, Qualität (z.B. Performanz)

Dann jeweils:

- Zusammengehörigkeit und Überlappungen von Anforderungen in einer großen Menge erkennen (z.B. von unterschiedlichen Autoren)
- Vollständigkeit der Anforderungen prüfen und
- Fehlende Anforderungen erkennen

Vorteil agiles Vorgehen: Software ist anders!

- Bei klassischen Produkten können Prototypen nicht einfach in MVPs überführt werden (Herstellung, Logistik, ...)
- Software kann einfach ausgeliefert werden und Komponenten enthalten, die Feedback der Nutzer einholen

Agenda

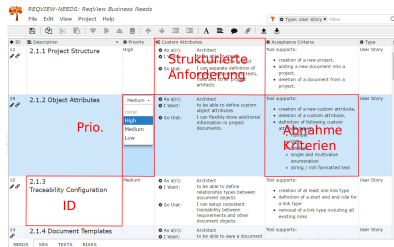
1. Motivation
2. Begriffe und Definitionen
3. Erheben von Anforderungen
4. Dokumentation von Anforderungen
5. Qualitätssicherung und Management
6. Management / Werkzeuge
7. Literatur und Referenzen

Anforderungen verwalten

Was verwalten:

- Anforderung
- Woher Anforderung kam (Autor, Betroffene)
 - Für wen ist sie wichtig
 - Vermeidet Phantome
 - Erlaubt gezielte Rückfragen
- Priorität
- Relation zu anderen Anforderungen
- Abnahmekriterien

Werkzeuge



- IBM Rational Doors
- PTC Integrity
- Vector PREEVision
- MS Excel
- ...

Priorisierung von Anforderungen

Wofür?

- Grundlage von technischen und Management-Entscheidungen
- Kompromisse zwischen in Konflikt stehenden Anforderungen finden
- Planung von Releases (zuerst die wichtigen Anforderungen)
- Zertifizierung in kontrollierten Domänen (z.B. Automotive)

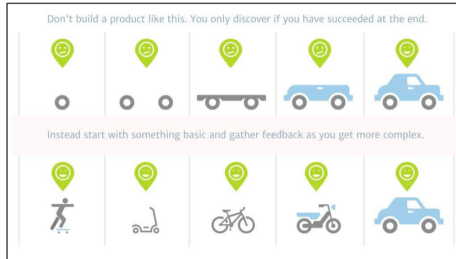
Beispiel

„**MUSS** bedeutet, dass es sich um eine absolut gültige und normative Festlegung bzw. Anforderung handelt.“

„**SOLL** beschreibt eine dringende Empfehlung. Abweichungen zu diesen Festlegungen sind in begründeten Fällen möglich. Wird die Anforderung nicht umgesetzt, müssen die Folgen analysiert und abgewogen werden.“

„**KANN** bedeutet, dass die Eigenschaften fakultativ oder optional sind. Diese Festlegungen haben keinen Normierungs- und keinen allgemeingültigen Empfehlungscharakter!“

Priorisierung: Minimal Viable Product



Minimal Viable Product = Kleinstes mögliches Produkt

- Schnell im Markt
- Schnell Feedback von Stakeholdern
- Kleineres Risiko
- Basis für Vertragsgestaltung

<https://thinkmobiles.com/blog/what-minimum-viable-product/>

Agenda

1. Motivation
2. Begriffe und Definitionen
3. Erheben von Anforderungen
4. Dokumentation von Anforderungen
5. Qualitätssicherung und Management
6. Management / Werkzeuge
7. Literatur und Referenzen

Zusammenfassung

Anforderungen

- Erwartungen von Beteiligten / Betroffenen an ein System und Entwicklungsprozess
- Basis für spätere Phasen der Entwicklung
- Basis für Verträge über Software Entwicklung
- Erfolgsfaktor in Projekten

Typen von Anforderungen

- Funktionale Anforderungen
- Qualitätsanforderungen
- Randbedingungen

Qualität von Anforderungen

- Korrektheit, Eindeutigkeit, Prüfbarkeit, Umsetzbarkeit, Nachverfolgbarkeit
- Minimalität, Vollständigkeit, Konsistenz, Erweiterbarkeit / Anpassbarkeit

Zusammenfassung

Erheben und Management von Anforderungen

- Betroffene identifizieren
- Vier Schritte: Erheben, Dokumentieren, Abstimmen, Verwalten
- Verschiedene Methoden zum Erheben: Beobachtung, Befragung, Recherche, ...
- Validieren: Qualität, Konsistenz, Korrektheit sicherstellen mit Hilfe von Reviews, Prototypen, ...
- Abstimmen: Priorisieren, Gruppieren und Klassifizieren

Dokumentation von Anforderungen

- Wichtig: Überprüfbarkeit von Anforderungen
- Verschiedene Methoden (von natürlicher Sprache bis zu vollständig formal)
 - Natürliche Sprache
 - Satzschablonen
 - Schablonen für Anwendungsfall
 - Anwendungsfalldiagramme
 - Weitere UML Diagramme
 - Formale Logik

Zusammenfassung

Inspiration: Design Thinking

- Software ist oft ein Design-Problem (Finden funktionaler Lösung)
- Design Thinking: Iteratives Vorgehen im Bereich Design
- Empathise / UX Research: In Nutzer einfühlen, Anforderungen erheben
 - vgl. Erheben von Anforderungen
 - Hier konkreter: Interviews, Fokusgruppen
- Prototype: Ideen für Lösungen früh ausprobieren und validieren
 - Verschiedene Ziele und Arten
- Test / Analytics: Anforderungen auf Produkt validieren
 - GQM und KPIs/Metriken
 - A/B Testing

- Pohl, Klaus, and Chris Rupp. Basiswissen Requirements Engineering : Aus- und Weiterbildung nach IREB-Standard zum Certified Professional for Requirements Engineering Foundation Level, dpunkt.verlag, 2015. ProQuest Ebook Central, Online lesbar [1]
- Sommerville, Ian. Software Engineering. 9. Aufl. Pearson, 2012. Kap. 4. isbn: 978-3868940992.
- Ebert, Christof. Systematisches Requirements Engineering. Anforderungen ermitteln, dokumentieren, analysieren und verwalten. 5. Aufl. dpunkt.verlag GmbH, 2014. isbn: 978-3-86490-139-3.

[1]: <http://ebookcentral.proquest.com/lib/dortmundtech/detail.action?docID=2029882>.

- Jens Jacobsen, Lorena Meyer: **Praxisbuch Usability und UX: Bewährte Usability- und UX-Methoden praxisnah erklärt**. 2., aktualisierte Auflage, Rheinwerk Computing, 2019. (über UB, Kopien von Auszügen)
 - Teil II
- Ralf Wirdemann, Johannes Mainusch: **Scrum mit User Stories**. 3., erweiterte Auflage, Hanser, 2017. (online über UB)
 - Kapitel 4
- Robert Kaiser: **Qualitative Experteninterviews Konzeptionelle Grundlagen und praktische Durchführung**. Springer, 2014. (online über UB)

Standards

- **IEEE Std 610.12-1990** Institute of Electric and Electronic Engineers: IEEE Standard Glossary of Software Engineering Terminology (IEEE Std 610.12-1990). IEEE Computer Society, New York, 1990.
- **IEEE Std 830-1998** Institute of Electric and Electronic Engineers: IEEE Recommended Practice for Software Requirements Specifications (IEEE Std 830-1998). IEEE Computer Society, New York, 1998.
- **ISO/IEC 25010:2011** International Organisation for Standardization: Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models, Genf 2011.
- **ISO/IEC/IEEE 29148:2011** International Organisation for Standardization: Systems and software engineering – Life cycle processes – Requirements engineering, Genf 2011.
- **OMG 2007** OMG: Unified Modeling Language: Superstructure, Version 2.1.1. OMG document formal/2007-02-05.
- **SEI 2006** Software Engineering Institute : CMMI for Development (CMMI-Dev), V1.2, Technical Report CMU/SEI-2006-TR-008 – ESC-TR-2006-008. Carnegie Mellon, Software Engineering Institute, Pittsburgh, PA 2006.